

1. Алфавит, грамматика и язык логики высказываний.

Логика высказываний, или пропозициональная логика (лат. *propositio* — «высказывание»), **или исчисление высказываний** — это раздел символической логики, изучающий сложные высказывания, образованные из простых, и их взаимоотношения.

Язык логики высказываний (пропозициональный язык) — искусственный язык, предназначенный для анализа логической структуры сложных высказываний.

Из конспекта: Язык логики высказываний – набор всевозможных текстов, описывающий логические формулы.

Язык логики высказываний **является формальным языком.**

Чтобы задать язык необходимо задать алфавит языка, синтаксические правила и семантические правила (Множество синтаксических правил **является грамматикой языка.** Семантические правила описывают механизм установки соответствия между каждым знаком в тексте языка и смыслом, значением этого знака).

Алфавит языка – это множество классов символов, из которых можно составлять тексты языка с помощью синтаксических правил.

Алфавит языка логики высказываний, разделены на следующие три категории:

- ⌚ пропозициональные буквы (пропозициональные переменные):

$p, q, r, s, t, p_1, q_1, r_1, s_1, t_1, \dots$

- ⌚ логические знаки (логические союзы):



Знак отрицания



Знак конъюнкции («И»)



Знак дизъюнкции («включающее ИЛИ»)



Знак строгой дизъюнкции («исключающее ИЛИ»)



Знак импликации



Знак эквивалентности

- ⌚ технические знаки:

(Левая скобка

) Правая скобка

Других знаков в алфавите языка логики высказываний нет.

2. Синтаксические виды формул языка логики высказываний.
Понятие подформулы языка логики высказываний, его свойства.

В логике высказываний используются формулы, а также такие понятия, как истина и ложь. Одной из основных задач математической логики и, в частности логики высказываний, является задача выяснения того, в каких тех или иных случаях истинна или ложна та или иная формула.

Формулы языка логики высказываний – правильно построенные выражения языка логики высказываний.

Определение:

1. Всякая пропозициональная переменная является формулой;
2. Если A - формула, то $\neg A$ также является формулой;
3. Если A и B - формулы, то выражения $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \not\rightarrow B)$, $(A \leftrightarrow B)$ также являются формулами;
4. Ничто иное не является формулой.

Формула, входящая в состав некоторой формулы, называется ее *подформулой*. В сложной формуле всегда можно выделить связку, которая называется ее *главным знаком*, – т.е. ту, которая связывает между собой подформулы.

Из конспекта:

Типология формул:

-Семантическая

Определяется на основе того, к какому классу относится формула

1. Общезначимые (все значения 1)
2. Противоречивые (все значения 0)
3. Нейтральные (хотя бы на одном 1 и хотя бы на одном 0)
4. Выполнимые (хотя бы одна 1)
5. Невыполнимые (все 0)

-Struct

1. СНФ
2. СДНФ
3. ДНФ
4. КНФ

° Формулы:

- Атомарная
- Не //

° Порф-ла

Ф-ла, в которой есть ^{только} переменные и const-атомарн, с лог. опер. — неатомарная.

Для неатом. ф-лы Φ ф-лы, которые связаны лог. опер. — порф-лы.

$\neg X_2$
 $\underbrace{\quad}_{\text{порф-ла}}$
 $\underbrace{\quad}_{\text{ф-ла}}$

$\underbrace{(a \vee b) \rightarrow c}_{\text{ф-ла}}$
 $\underbrace{\quad}_{\text{порф-ла}} \quad \underbrace{\quad}_{\text{порф-ла}}$
 $\underbrace{\quad}_{\text{порф-ла}} \quad \underbrace{\quad}_{\text{порф-ла}}$

3. Понятие интерпретации формулы языка логики высказываний. Число интерпретаций формулы.

Определение 10 (Интерпретация). Символы л,и ("ложь", "истина") называются *истинностными значениями*. Интерпретация пропозициональной сигнатуры Σ есть функция из Σ в $\{л,и\}$.

Если Σ конечна, тогда интерпретация может быть определена таблицей её значений, например:

p	q
$л$	$и$

(3
)

Семантика логики высказываний, которую мы собираемся ввести, определяет какие истинностные значения назначены формуле F интерпретацией I .

Прежде всего нам надо связать функцию с каждой пропозициональной связкой – функцию из $\{л,и\}$ в $\{л,и\}$ с унарной связкой $\neg$ и функцию из $\{л,и\} \times \{л,и\}$ в $\{л,и\}$ с каждой бинарной связкой. Функции определяются следующими таблицами:

x	$\neg(x)$	x	y	$\&(x, y)$	$\text{Б}(x, y)$	$\text{И}(x, y)$
$л$	$и$	$л$	$л$	$л$	$л$	$и$
$л$	$и$	$л$	$и$	$л$	$и$	$и$
$и$	$л$	$и$	$л$	$л$	$и$	$л$
$и$	$л$	$и$	$и$	$и$	$и$	$и$

Для любой формулы F и любой интерпретации I истинностное значение F^I , назначенное формуле F интерпретацией I , определяется как значение суперпозиции соответствующих булевых функций, а именно, следующим образом:

- ⌚ $F^I = I(F)$ если F - атом,
- ⌚ $(\neg F)^I = \neg(F^I)$,
- ⌚ $(F \rightarrow G)^I = \rightarrow(F^I, G^I)$ для каждой бинарной связки $\rightarrow$.

Заметим, что это определение рекурсивно:
 $(\neg F)^I$ определяется через F^I и $(F \rightarrow G)^I$ - через F^I и G^I .

Если $F^I = \text{и}$, мы говорим, что формула F истинна при интерпретации I (символически $I \models F$).

2.10 Найдите формулу F такую, что (3) - единственная интерпретация, при которой F истинна.

Если рассматриваемая сигнатура конечна, тогда множество интерпретаций тоже конечно, и значения F^I для всех интерпретаций можно представить в виде конечной таблицы. Эта таблица называется *таблицей истинности* формулы F . Например, предыдущая задача может быть сформулирована следующим образом: найти формулу, таблицей истинности которой является

p	q	
л	л	л
л	и	и
и	л	л
и	и	л

4. Семантические виды формул языка логики высказываний. Нейтральность, невыполнимость, общезначимость

Синтаксис логики высказываний прост и имеет прямые синтаксические и семантические аналоги в естественных языках, что чрезвычайно облегчает нам понимание логики высказываний. Символами языка логики высказываний, составляющими ее алфавит, являются логические константы ИСТИНА и ЛОЖЬ, сокращенно обозначаемые буквами И и Л, логические переменные x, y, z , обозначаемые строчными буквами латинского алфавита, логические связки (И), (ИЛИ), (НЕ), ЭКВИВАЛЕНТНО, $\Rightarrow$ (ВЛЕЧЕТ) и круглые скобки. Значениями логических переменных являются логические константы. Предложения языка логики высказываний, называемые также формулами или высказываниями, составляют в соответствии со следующими правилами:

Логические константы являются простыми предложениями;

Логические переменные также простые предложения;

Сложные предложения формируются из простых с помощью связок (И), (ИЛИ), (НЕ), (ЭКВИВАЛЕНТНО), $\Rightarrow$ (ВЛЕЧЕТ);

Простые и сложные предложения, заключенные или не заключенные в скобки, являются предложениями языка логики высказываний;

Из предложений с помощью связок и скобок можно

образовать новые предложения языка логики высказываний;

Связки имеют следующий порядок старшинства $\emptyset, \cup, \cap, \rightarrow, \leftrightarrow$, т.е. связка $\emptyset$ самая старшая, а связка $\leftrightarrow$ самая младшая.

Формулы логики высказываний, составленные по этим правилам, называют **правильно построенными формулами** или сокращенно **формулами**.

2.1.2. Семантика логики высказываний.

Семантику логики высказываний можно пояснить **смысловой интерпретацией ее предложений** или формул, под которой обычно понимают процесс установления соответствия между логическими переменными и изменяющимися свойствами объектов среды и между значениями переменных (константами) и конкретными значениями свойств объектов. С помощью алгебры логики можно записывать любое правило и предложение для СЛ.

Пример 1. Студент, который не ходит на занятия и не занимается дома, не сдаст экзамен. Обозначим через x_1 событие: ходить на занятия; через x_2 событие – заниматься дома и через y – сдать экзамен. Тогда формальная запись на языке алгебры логики будет иметь вид:

$$\neg y = \neg x_1 \cup \neg x_2$$

Пример 2. Если истинно утверждение, что Иван и Мария являются родителями и Юра и Анна, то Юра и Анна являются братом и сестрой. Обозначим: x_1 – Иван, x_2 – Мария, x_3 – Юра, x_4 – Анна, y_1 – родители Юры, y_2 – родители Анны, y – Юра и Анна – брат и сестра.

$$y_1 = x_1 \cap x_2, y_2 = x_1 \cap x_2, y = y_1 \cap y_2.$$

То же самое можно описать словами:

“Родители_Юры” = “Иван” $\cap$ “Мария”

“Родители_Анны” = “Иван” $\cap$ “Мария”

“Юра_и_Мария

брат_и_сестра” = “Родители_Юры” $\cap$ “Родители_Анны”

Иначе говоря, интерпретация определяет семантику формул (предложений, высказываний) путем сопоставления переменных в формулах со свойствами объектов среды, а отношений между этими свойствами — с формулами. Это позволяет по значению формул после подстановки вместо переменных конкретных значений свойств судить о наличии или отсутствии у среды тех или иных совокупных свойств или отношений. Если дана какая-либо формула, то подстановка в формулу констант вместо ее переменных называется **конкретизацией**. Таким образом конкретизация является результатом интерпретации.

Будем полагать, что, употребляя формулу $x \dot{E} y$, мы вкладываем в нее смысл, вытекающий из следующего предложения: **«Мы заявляем, что истинность высказывания « x влечет y » означает, что истинность x влечет истинность y , а больше мы ничего не заявляем».**

Истинностные значения любой формулы, т.е. ее семантику, всегда можно задать таблицей, состоящей из двух частей: в левой части таблицы перечислены все наборы значений аргументов, а в правой соответствующие наборам значения формулы. Задание таких таблиц для связок облегчается тем, что значениями аргументов и формул являются только две величины – И или Л. Такие таблицы в логике высказываний называют **таблицами истинности**.

Если формула интерпретирована, то ее таблица истинности определяет семантику интерпретированной формулы, поскольку по ней можем всегда определить, какие же отношения между свойствами объектов, обозначаемых переменными, имеют место (формула истинна) и не имеют места (формула ложна).

С помощью алгебры логики можно записывать любое правило и предложение для СИИ.

2.1.3. Общезначимые формулы и их роль.

Формулы, истинные на всех наборах значений своих аргументов, называют **общезначимыми формулами**. Если какая-либо формула A является общезначимой, то этот факт обычно записывается с использованием знака

общезначимости $\frac{1}{2} =$ который ставится перед формулой: $\models a$. Проверку формулы на общезначимость можно осуществить с помощью таблицы истинности: если формула истинна во всех строках таблицы истинности, то эта формула общезначима. Рассмотрим, например, формулу $\forall x \neg (\exists y \neg y) \rightarrow x \neg x$. Из таблицы ясно, что формула является общезначимой, т.к. значения переменных в последнем столбце всегда истинно.

Ранее было заявлено, что истинность высказывания $a_1 \rightarrow a_2$ всегда означает, что истинность a_1 влечет истинность a_2 (здесь a_1 и a_2 являются формулами логики высказываний). Потому, установив факт общезначимости формулы $a_1 \rightarrow a_2$ и истинности a_1 , всегда можно сделать заключение об истинности a_2 . Таким образом, общезначимость формул вида $a_1 \rightarrow a_2$, называемых имплицативными формулами, является важным свойством для получения заключения об истинности a_2 , называемого **заключением**, при истинности a_1 , называемого **посылкой**. Для простоты имплицативные формулы $a_1 \rightarrow a_2$ будем называть так же, как и связку $\rightarrow$, импликацией. В логике высказываний известно много общезначимых формул, называемых обычно законами логики высказываний. Наиболее известными являются следующие законы.

Коммутативные: $x_1 \rightarrow x_2 = x_2 \rightarrow x_1$, $x_1 \vee x_2 = x_2 \vee x_1$;

Дистрибутивные: $x_1 \rightarrow (x_2 \wedge x_3) = (x_1 \rightarrow x_2) \wedge (x_1 \rightarrow x_3)$,
 $x_1 \vee (x_2 \rightarrow x_3) = (x_1 \vee x_2) \rightarrow (x_1 \vee x_3)$;

Ассоциативные: $x_1 \rightarrow (x_2 \rightarrow x_3) = (x_1 \rightarrow x_2) \rightarrow x_3$, и
 $x_1 \vee (x_2 \vee x_3) = (x_1 \vee x_2) \vee x_3$;

законы Де Моргана:

$$\overline{(x \wedge x_2)} = \bar{x} \vee \bar{x}_2, \quad \overline{(x_1 \vee x_2)} = \bar{x}_1 \wedge \bar{x}_2,$$

закон двойного отрицания: $\overline{\bar{x}} = x$.

Кроме того, справедливы следующие соотношения:

$$x \wedge 0 = 0, \quad x \wedge 1 = x, \quad x \wedge \bar{x} = 0, \quad x \vee 1 = 1, \quad x \vee 0 = x, \quad x \vee \bar{x} = 1.$$

Сложные формулы, как правило, можно упростить. Для

этого можно использовать следующие эквивалентности:

$$x_1 \vee x_1 \wedge x_2 = x_1,$$

$$x_1 \wedge (x_1 \vee x_2) = x_1 \text{ – правила поглощения,}$$

$$x_1 \wedge x_2 \vee x_1 \wedge \bar{x}_2 = x_1 \text{ – правило склеивания,}$$

$$x_1 \wedge x_2 \vee \bar{x}_1 = x_2 \vee \bar{x}_1 \text{ – правило вычеркивания.}$$

Их доказательства осуществляются путем построения соответствующих таблиц истинности.

Рассмотрим пример упрощения логической функции. Пусть

$$f = \overline{((x_1 \wedge x_2) \vee x_3) \wedge (1 \vee (\bar{x}_3 \wedge x_2))}$$

Последовательное применение приведенных выше правил дает:

$$\begin{aligned} f &= \overline{((x_1 \wedge x_2) \vee x_3) \wedge (1 \vee (\bar{x}_3 \wedge x_2))} = \overline{((x_1 \wedge \bar{x}_2) \vee x_3)} \vee \overline{(1 \vee (\bar{x}_3 \wedge x_2))} = \\ &= (\overline{x_1 \wedge \bar{x}_2} \wedge \bar{x}_3) \vee (1 \vee (\bar{x}_3 \wedge x_2)) = (\bar{x}_1 \vee x_2 \wedge \bar{x}_3) \vee 1 = 1. \end{aligned}$$

Кроме общезначимых, существуют формулы выполнимые и невыполнимые. Формула называется **выполнимой**, если существуют наборы значений ее аргументов, на которых она принимает истинное значение, и наборы значений, на которых она принимает ложное значение.

Если формула на всех наборах значений ее аргументов принимает ложное значение, то она называется невыполнимой.

Установление истинности следствия по общезначимой имплицативной формуле достаточно универсальный способ для вывода заключений, но требует проверки общезначимости последней. **Если формула $a_1 \rightarrow a_2$ не является общезначимой, то подобного заключения делать нельзя.**

Проверку общезначимости можно осуществить с помощью таблицы истинности. Однако построение таблиц истинности слишком трудоемко для того, чтобы можно было решать

реальные задачи. Вместо этого используют специальные правила вывода, применение которых базируется не на понятии общезначимости формулы, в частности общезначимости импликативной формулы, а на понятии модели формулы.

5. Логическое следование формул (отношение логического следования формул)

Одна формула следует из другой тогда и только тогда, когда для любой интерпретации, для которой значение обозначаемой второй формулой функции равнозначно 1, значение обозначаемой первой формулой функции также равнозначно 1. Например, формула $(P \rightarrow Q)$ следует из формулы $(P \wedge Q)$. Это записывается так $((P \wedge Q) \rightarrow (P \rightarrow Q))$. Если формула следует из любой формулы, т.е. является общезначимой, то это записывается так: $(\vdash (P \rightarrow P))$. Формула следует из формулы тогда и только тогда, когда множество невыполнимо.

Формула G называется *логическим следствием* формулы F (логически следует из формулы F), пишут $F \models G$, если для любой интерпретации α переменных в F, G , для которой $F(\alpha) = 1$, имеет место и $G(\alpha) = 1$. Обобщение:

Логическое следование. Формула $G(X_1, X_2, \dots, X_n)$ называется *логическим следствием* формул $F_1(X_1, X_2, \dots, X_n), \dots, F_m(X_1, X_2, \dots, X_n)$, если она обращается в истинное высказывание на всяком наборе значений переменных, для которого в истинные высказывания обращаются все формулы $F_1, \dots, F_m$. Это обозначается: $F_1, \dots, F_m \models G$.

Формулы $F_1, \dots, F_m$ называются *посылками* для логического следствия H .

Таким образом, $F_1, \dots, F_m \models H$, если для любых высказываний $A_1, \dots, A_n$ из $\lambda(F_1(A_1, \dots, A_n)) = 1, \dots, \lambda(F_m(A_1, \dots, A_n)) = 1$ следует $\lambda(H(A_1, \dots, A_n)) = 1$. Наконец можно и так сказать о логическом следствии. Составим таблицы истинности для формул $F_1, \dots, F_m, H$. Предположим, что если в какой-то строке таблицы все формулы $F_1, \dots, F_m$ принимают значение 1, то в этой строке непременно и формула H принимает значение 1. Это и будет означать, что H является логическим следствием формул $F_1, \dots, F_m$.

Признак логического следствия:

Формула H будет

логическим следствием формулы F тогда и только тогда, когда формула $F \rightarrow H$ является тавтологией: $F \models H \Leftrightarrow \models F \rightarrow H$.

6. Равносильность формул (отношение равносильности)

Две формулы равносильны тогда и только тогда, когда следуют друг из друга. Например, формулы $(\neg P \vee Q)$ и $(P \rightarrow Q)$ равносильны, так как: $((\neg P \vee Q) \rightarrow (P \rightarrow Q))$ и $((P \rightarrow Q) \rightarrow (\neg P \vee Q))$. Равносильность формул записывается так: $((P \rightarrow Q) \leftrightarrow (\neg P \vee Q))$. Следовательно, две формулы равносильны тогда и только тогда, когда обозначают функции, для которых существует каждой из них равнозначная или равная функция.

Формулы $F(X_1, X_2, \dots, X_n)$ и $H(X_1, X_2, \dots, X_n)$ алгебры высказываний называются *равносильными (эквивалентными)*, если при любых значениях входящих в них пропозициональных переменных логические значения получающихся из формул F и H высказываний совпадают. Для указания равносильности формул используют обозначение $F \equiv H$. Определение равносильности формул можно записать символически:

$$F \equiv H \Leftrightarrow \lambda(F(A_1, A_2, \dots, A_n)) = \lambda(H(A_1, A_2, \dots, A_n))$$

для любых конкретных высказываний $A_1, A_2, \dots, A_n$.

Чтобы проверить равносильность двух формул F и G , надо построить таблицу истинности обеих формул и убедиться, что значения истинности получающихся из них высказываний одинаковы для любых одинаковых наборов значений пропозициональных переменных.

Признак равносильности формул:

Две формулы F и G равносильны тогда и только тогда, когда их эквивалентность $F \leftrightarrow G$ является тавтологией.

Отношение равносильности формул рефлексивно, симметрично и транзитивно, т.е. является отношением эквивалентности.

Справедливы следующие равносильности:

$$\neg \neg P \equiv P;$$

$$P \rightarrow Q \equiv \neg Q \rightarrow \neg P;$$

$$P \leftrightarrow Q \equiv \neg P \leftrightarrow \neg Q;$$

$$P \rightarrow (Q \rightarrow R) \equiv (P \wedge Q) \rightarrow R;$$

$$(P \rightarrow R) \wedge (Q \rightarrow R) \equiv (P \vee Q) \rightarrow R;$$

$P \wedge P \equiv P;$
 $P \vee P \equiv P;$
 $P \wedge Q \equiv Q \wedge P;$
 $P \vee Q \equiv Q \vee P;$
 $P \wedge (Q \wedge R) \equiv (P \wedge Q) \wedge R;$
 $P \vee (Q \vee R) \equiv (P \vee Q) \vee R;$
 $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R);$
 $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R);$
 $P \wedge (P \vee Q) \equiv P;$
 $P \vee (P \wedge Q) \equiv P;$
 $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ (1-й закон де Моргана);
 $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ (2-й закон де Моргана);
 $P \leftrightarrow Q \equiv Q \leftrightarrow P;$
 $P \rightarrow Q \equiv \neg P \vee Q;$
 $P \rightarrow Q \equiv \neg(P \wedge \neg Q);$
 $P \wedge Q \equiv \neg(P \rightarrow \neg Q);$
 $P \vee Q \equiv \neg P \rightarrow Q;$
 $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P);$
 $P \vee \neg P \equiv 1, P \wedge \neg P \equiv 0;$
 $P \vee 1 \equiv 1, P \wedge 1 \equiv P;$
 $P \vee 0 \equiv P, P \wedge 0 \equiv 0.$

Лемма. Если в формуле некоторую ее подформулу заменить на равносильную ей формулу, то полученная формула будет равносильна исходной:

Если $G(Y_1, \dots, Y_s) \equiv H(Y_1, \dots, Y_s)$, то для любой формулы алгебры высказываний $F(X_1, \dots, X_{i-1}, Y, X_{i+1}, \dots, X_n)$ имеет место равносильность $F(X_1, \dots, X_{i-1}, G(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n) \equiv F(X_1, \dots, X_{i-1}, H(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n)$.

Доказательство. Поскольку формулы $G(Y_1, \dots, Y_s)$ и $H(Y_1, \dots, Y_s)$ принимают всегда одинаковые значения при одинаковых значениях пропозициональных переменных $Y_1, \dots, Y_s$, то формулы

$$F(X_1, \dots, X_{i-1}, G(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n)$$

$$\text{и } F(X_1, \dots, X_{i-1}, H(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n)$$

принимают одинаковые значения при любых одинаковых наборах значений переменных $X_1, \dots, X_n, Y_1, \dots, Y_s$. Следовательно,

$$\equiv F(X_1, \dots, X_{i-1}, G(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n) \leftrightarrow$$

$$\leftrightarrow F(X_1, \dots, X_{i-1}, H(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n),$$

т.е. $F(X_1, \dots, X_{i-1}, G(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n) \equiv F(X_1, \dots, X_{i-1}, H(Y_1, \dots, Y_s), X_{i+1}, \dots, X_n)$, что и требовалось доказать. $\square$

Данная лемма служит основанием для упрощения формул путем равносильных преобразований (т.е. переходя от одной формулы к равносильной ей).

Равносильные преобразования также применяются для приведения формул к специальной форме (виду).

7. Алгебра высказываний и истинностные функции.

Рассмотрим двуэлементное множество $(B = \{\perp, T\})$.

Будем называть n -арной операцией на множестве A функцию ϕ вида

$$\phi \in A^{(A^n)}.$$

Истинностной n -местной функцией будем называть любую функцию являющуюся подмножеством n -арной операции ϕ вида $\phi \in B^{(B^n)}$.

Алгебру высказываний задают парой, где первый компонент – множество всевозможных истинностных функций, а второй компонент – некоторое множество операций, являющихся высказывательными функциями.

8. Базис истинностных функций. Булева алгебра высказываний.

Логическим базисом называется множество M тогда только тогда, когда оно является полной системой истинностных функций и ни одно его собственное строгое подмножество не является полной системой истинностных функций.

Полная система истинностных функций (на всякий) – множество E тогда и только тогда, когда его значение по суперпозиции равно F , где F – множество N -арных истинностных функций.

Булева алгебра

Если в соответствующий логический базис выбрать операции, которые удовлетворяют условиям ниже, то можно говорить о булевой алгебре.

Условия:

Ассоциативность, дистрибутивность, коммутативность, идемпотентность, двойное отрицание, правила де Моргана.

Вопрос 11. Формальная теория логики высказываний. Правила вывода, теоремы и формальный вывод.

Формальная аксиоматическая теория (исчисление) – подмножеством формул некоторого языка. Обычно это множество включает только общезначимые формулы. Исчисление задаётся четырьмя компонентами (вектора):

- алфавитом языка,
- грамматикой языка,
- множеством формул, называемых аксиомами,
- множеством правил (логического) вывода.

Правила вывода

Над формулами исчисления с помощью правил вывода задаётся отношение выводимости ((логического) вывода) $\vdash$. Рассмотрим следующее правило вывода, которое называется правилом прямого заключения или правилом Modus Ponens и описано на метаязыке, в следующей формулировке.

Для любого множества G $G \subseteq L$ и для любых формул α и β языка L , выполняется следующее:
кроме вышеприведённого свойства отношение вывода является транзитивным отношением и обладает следующим свойством:

где G и F произвольные множества формул языка L .

Формальный (логический) вывод формулы β из множества посылок G – вектор (последовательность) логических формул, каждый i -й компонент которого является аксиомой или элементом множества G , либо – формулой, которая выводима (получена путём применения правила вывода) из множества формул, состоящего из всех таких формул и только таких формул, которые являются k -ми компонентами этого вектора,

где ($k < i$) , а формула β является последним компонентом этого вектора. Формула β является теоремой (теоремой исчисления) тогда и только тогда, когда существует формальный вывод формулы β из множества посылок, являющегося подмножеством аксиом этого исчисления.

Следующие обозначения отношения выводимости эквивалентны:

Пример формального вывода (доказательства) теоремы ($P \rightarrow P$) в заданном исчислении высказываний.

Это означает, что первая и третья формулы получены из аксиомной схемы введения импликации, вторая – из аксиомной схемы удаления импликации, четвёртая и пятая – по правилу прямого заключения, используя соответственно пары формул $\langle 1, 2 \rangle$ и $\langle 3, 4 \rangle$.

Вопрос 12. Формальная теория логики высказываний.
Отношение выводимости, его свойства.

Формальная аксиоматическая теория (исчисление) – подмножеством формул некоторого языка. Обычно это множество включает только общезначимые формулы. Исчисление задаётся четырьмя компонентами (вектора):

- алфавитом языка,
- грамматикой языка,
- множеством формул, называемых аксиомами,
- множеством правил (логического) вывода.

Метатеоремы

Кроме теорем исчисления могут рассматриваться некоторые формулы метаязыка, описывающие закономерности между произвольными формулами и отношениями для этого исчисления. Такие формулы включают метаварьируемые и в случае – когда могут быть доказаны – называются метатеоремами. Метатеоремы могут формулировать свойства отношения выводимости, дополнительные правила вывода или другие закономерности. Некоторые метатеоремы могут оказаться теоремами некоторого другого исчисления. Рассмотрим метатеоремы, формулирующие следующие свойства отношения выводимости для любых множеств формул G , F и E языка L .

13 Связь тождественно-истинных высказываний и теорем исчисления высказываний, формальной теории высказываний

Тождественно-истинные высказывания - высказывания, выражения или формулы логических исчислений, являющиеся истинными при любых значениях истинности их переменных.

Методы проверки тождественной истинности формул

1. С помощью таблиц истинности, когда приходится вычислять значений формулы во всех интерпретациях.
2. Алгебраический метод, который базируется на применении законов булевой алгебры. В этом случае мы упрощаем формулу до тех пор, пока она либо не станет равна 1, либо 0, либо мы не сведем к простой формуле (например, МинДНФ), по которой видно, что это не тавтология.

3. Метод Квайна. В этом случае, разлагаем формулу R по первой переменной, придавая значение 0 и 1, получая более простые формулы R_0 и R_1 . Затем разлагаем их по второй переменной и т.д. Возможно, на каком-то шаге, мы увидим тавтологию или противоречие. В общем случае, метод Квайна требует меньшую трудоемкость, чем использование таблиц истинности с их полным перебором возможных значений переменных.

14 Метатеорема дедукции исчисления высказываний. Обратная теорема.

Метатеорема - теорема относительно объектов (понятий, определений, аксиом, доказательств, правил вывода, теорем и др.) какой-либо научной теории, доказываемая средствами метатеории этой теории. Термин «метатеорема» употребляется преимущественно в применении к теоремам об объектах формализованных теорий (т. е. в случае, когда предметная теория является исчислением, или формальной системой).

$$G_1, G_2, G_3, G_4 \vdash R$$

$$G_1, G_2, G_3 \vdash G_4 \Rightarrow R$$

$$G_1, G_2 \vdash G_3 \Rightarrow (G_4 \Rightarrow R)$$

$$G_1 \vdash G_2 \Rightarrow (G_3 \Rightarrow (G_4 \Rightarrow R))$$

$$\vdash G_1 \Rightarrow (G_2 \Rightarrow (G_3 \Rightarrow (G_4 \Rightarrow R)))$$

Метатеорема дедукции

(МТД): $G_1, G_2, \dots, G_{\beta-1}, G_{\beta} \vdash R$, то можно составить доказательство $G_1, G_2, \dots, G_{\beta-1} \vdash G_{\beta} \Rightarrow R$. Проще говоря, значок можно переносить справа налево, на место запятой, оставляя вместо него на каждом шаге символ $\Rightarrow$

Обратное движение также допустимо. При движении значка вправо символ $\vdash$ должен ставиться на место операции $\Rightarrow$, которая обрабатывается (согласно скобкам

и приоритетам) самой последней. Если в формуле последней обрабатывается другая операция, то дальнейшее движение значка невозможно. При движении влево надо не забывать расставлять скобки, чтобы в образовавшейся формуле новый символ $\Rightarrow$ обрабатывался (согласно скобкам и приоритетам) самым последним.

Формально описывается так:

Теорема дедукции

$$(((G \cup \{\alpha\}) \vdash \{\beta\}) \rightarrow (G \vdash \{(\alpha \rightarrow \beta)\}))$$

Теорема обратная теореме дедукции

$$((G \vdash \{(\alpha \rightarrow \beta)\}) \rightarrow ((G \cup \{\alpha\}) \vdash \{\beta\}))$$

, где G – множество формул языка, a, b – формулы языка

15) Правило обратного логического заключения (Modus Tollens).

Modus tollens — рассуждение от противного

Схема рассуждения:

Если A , то B ; неверно B .

Неверно А.

Здесь А и В - некоторые высказывания; "если А, то В" и "неверно, что В" ("не-В") - посылки; "неверно, что А" ("не-А") - заключение; горизонтальная черта стоит вместо слова "следовательно". Другая запись:

Если А, то В. Не-В. Следовательно, не-А.

Посредством этой схемы от утверждения условного высказывания и отрицания его следствия (консеквента) осуществляется переход к отрицанию основания (антецедента) данного высказывания. Напр.: Если гелий - металл, он электропроводен.

Гелий неэлектропроводен.

Гелий - не металл.

16) Противоречивая формальная теория. Резолюция в логике высказываний.

Формальная система (формальная теория, аксиоматическая теория, дедуктивная система) — результат строгой формализации теории, предполагающей полную абстракцию от смысла слов используемого языка, причем все условия, регулирующие употребление этих слов в теории, явно высказаны посредством аксиом и правил, позволяющих вывести одну фразу из других.

Формальная система — это совокупность абстрактных объектов, не связанных с внешним миром, в которой представлены правила оперирования множеством символов в строго синтаксической трактовке без учёта смыслового содержания, то есть семантики.

Теория, в которой множество теорем покрывает всё множество формул (все формулы являются теоремами, «истинными высказываниями»), называется *противоречивой*. Выяснение противоречивости теории — одна из важнейших и иногда сложнейших задач формальной логики. После выяснения противоречивости теория, как правило, не имеет дальнейшего ни теоретического, ни практического применения.

Правило резолюций — это правило вывода, восходящее к методу доказательства теорем через поиск противоречий; используется в логике высказываний и логике предикатов первого порядка. Правило резолюций, применяемое последовательно для списка резольвент, позволяет ответить на вопрос, существует ли в исходном множестве логических выражений противоречие.

Алгоритмы доказательства выводимости $A \vdash B$, построенные на основе этого метода, применяются во многих системах искусственного интеллекта, а также являются фундаментом, на котором построен язык логического программирования «Пролог».

Пусть C_1 и C_2 — два предложения в исчислении высказываний, и пусть $C_1 = P \vee C'_1$, а $C_2 = \neg P \vee C'_2$, где P — пропозициональная переменная, а C'_1 и C'_2 — любые предложения (в частности, может быть, пустые или состоящие только из одного литерала).

Правило вывода

$$\frac{C_1, C_2}{C'_1 \vee C'_2}$$

называется **правилом резолюций**.

Метод резолюции

Доказательство теорем сводится к доказательству того, что некоторая формула G (гипотеза теоремы) является логическим следствием множества формул $F_1, \dots, F_k$ (допущений). То есть сама теорема может быть сформулирована следующим образом: «если $F_1, \dots, F_k$ истинны, то истинна и G ».

Для доказательства того, что формула G является логическим следствием множества формул $F_1, \dots, F_k$, метод резолюций применяется следующим образом. Сначала составляется множество формул $\{F_1, \dots, F_k, \neg G\}$. Затем каждая из этих формул приводится к КНФ (конъюнкция дизъюнктов) и в полученных формулах зачеркиваются знаки конъюнкции. Получается множество дизъюнктов S . И, наконец, ищется вывод пустого дизъюнкта из S . Если пустой дизъюнкт выводим из S , то формула G является логическим следствием формул $F_1, \dots, F_k$. Если из S нельзя вывести $\#$, то G не является логическим следствием формул $F_1, \dots, F_k$. Такой метод доказательства теорем называется *методом резолюций*.

Рассмотрим пример доказательства методом резолюций. Пусть у нас есть следующие утверждения:

«Яблоко красное и ароматное.»

«Если яблоко красное, то яблоко вкусное.»

Докажем утверждение «яблоко вкусное». Введем множество формул, описывающих простые высказывания, соответствующие вышеприведенным утверждениям. Пусть

X_1 — «Яблоко красное»,

X_2 — «Яблоко ароматное»,
 X_3 — «Яблоко вкусное».

Тогда сами утверждения можно записать в виде сложных формул:

$X_1 \ \& \ X_2$ — «Яблоко красное и ароматное.»
 $X_1 \rightarrow X_3$ — «Если яблоко красное, то яблоко вкусное.»

Тогда утверждение, которое надо доказать, выражается формулой X_3 .

Итак, докажем, что X_3 является логическим следствием $(X_1 \ \& \ X_2)$ и $(X_1 \rightarrow X_3)$. Сначала составляем множество формул с отрицанием доказываемого высказывания; получаем:

$\{(X_1 \ \& \ X_2), (X_1 \rightarrow X_3), \neg X_3\}$.

Теперь приводим все формулы к конъюнктивной нормальной форме и зачеркиваем конъюнкции. Получаем следующее множество дизъюнктов:

$\{X_1, X_2, (\neg X_1 \vee X_3), \neg X_3\}$.

Далее ищем вывод пустого дизъюнкта.

Применяем к первому и третьему дизъюнктам правило резолюции:

$\{X_1, X_2, (\neg X_1 \vee X_3), \neg X_3, X_3\}$.

Применяем к четвёртому и пятому дизъюнктам правило резолюции:

$\{X_1, X_2, (\neg X_1 \vee X_3), \neg X_3, X_3, \#\}$.

Таким образом пустой дизъюнкт выведен, следовательно выражение с отрицанием высказывания опровергнуто, следовательно само высказывание доказано.

17. Алфавит, грамматика и язык логики предикатов первого порядка.

В логике высказываний атомарным выражением законченной мысли является непосредственно высказывание, рассматриваемое как единое (нераздельное) целое. Ни структура высказывания, ни его содержание не затрагиваются. В то же время и в науке, и на практике используются выражения, существенным образом зависящие как от структуры, так и от содержания используемых в них высказываний.

В связи с этим возникает необходимость в расширении логики высказываний, в построении такой логической системы, средствами которой можно было бы исследовать структуру высказываний. Такой логической системой является логика предикатов (лат. *praedicatum* - заявленное, упомянутое, сказанное).

Логика предикатов расчленяет высказывание на объект (подлежащее) и предикат (сказуемое). **Объект** - это то, о чем что-то утверждается в высказывании; **предикат** - это то, что утверждается об объекте.

Достоинства логики предикатов первого порядка :

- в качестве «фундамента» используется классический аппарат математической логики, методы которой достаточно хорошо изучены и формально обоснованы;

- в базах знаний можно хранить лишь множество аксиом, а все остальные знания получать из них по правилам вывода.

Недостатки логики предикатов первого порядка - требуется адекватное описание предметной области, т.е. в базе знаний должна быть представлена вся информация, необходимая для решения задачи;

- невозможность применения в качестве термов (параметров) предикатов других предикатов, т.е. невозможность формулирования знаний о знаниях.

Синтаксис логики предикатов первого порядка - логические константы – ИСТИНА (И, TRUE, T) и ЛОЖЬ (Л, FALSE, F);

- символы констант – символьные выражения, начинающиеся со строчной буквы (например, cat, blue);

- символы переменных – символьные выражения, начинающиеся с прописной буквы или знака подчеркивания (например, X, Cat, _blue);

- функциональные и предикатные символы – символьные выражения, начинающиеся со строчной буквы, за которыми следуют список аргументов (термов), заключенных в скобки (например, кошка(катя), sin(X), друзья(вася, петя)). Отличие функции от предиката, заключается в том, что функция возвращает результаты любого типа (строку, число, дату и т.д.), а предикат – только логического типа (истину или ложь). В связи с этим предикаты иногда называют логическими функциями;

- логические символы (пропозициональные связки):

- ➤ – отрицание;

- ▲ – логическое И (конъюнкция, логическое умножение);

- ▼ – логическое ИЛИ (дизъюнкция, логическое сложение);

- * (⊃) – импликация (если - то);

- $\star (\odot)$ – эквивалентность;
- кванторы переменных:
 - $\exists$ – квантор существования;
 - $\forall$ – квантор всеобщности;
- вспомогательные символы: круглые скобки, запятые, $+$, $-$ и т.п.

Семантика логики предикатов первого порядка (основные определения).

Терм - переменная, константа или функция (т.е. объект или его свойство).

18.Предикат. Модель, теоретико-множественные модельные интерпретации формул логики предикатов. Число интерпретаций.

Предикатом n аргументов (n -местным предикатом) будем называть любую функцию являющуюся подмножеством n -арной операции ϕ вида $\phi \in B, A$, где A – произвольное множество и $(B = \{\perp, \cdot\})$. Любой предикат двух и более аргументов характеризует отношение и называется характеристической функцией отношения.

Если предикат, характеризующий отношение, истинен для какого-либо аргумента, то этот аргумент принадлежит этому отношению, и если – ложен, то – не принадлежит

Интерпретация формул логики предикатов

Формулы логики высказываний всегда можно рассматривать как высказывательные формы с высказывательными переменными либо как высказывания. Формулы логики предикатов становятся высказывательными формами с предметными переменными или высказываниями, если задать непустое множество M значений, которые можно приписывать предметным переменным, входящим в формулу, а каждому n -местному предикатному символу поставить в соответствие n -местный предикат, определенный на множестве M (причем двум различным n -местным предикатным символам с одинаковыми предикатными буквами ставится в соответствие один и тот же предикат); нуль-местным предикатным символам независимо от выбора множества M приписывается нуль-местный предикат, т.е. одно из значений истинности $\{И, Л\}$.

Если формула не содержит свободных предметных переменных, то, задав множество M и приписав предикатным символам конкретные предикаты, мы получим высказывание (точнее говоря, значение истинности). Если же в формуле есть свободные вхождения предметных переменных, то получим высказывательную форму от этих переменных, которая станет высказыванием, если подставить вместо свободных вхождений переменных элементы множества M .

Обращение формулы в высказывание описанным выше способом будем называть **интерпретацией** этой формулы.

19. Логические кванторы, их семантика и взаимные свойства.

Кванторы (кванторные формулы) являются способом сокращённой записи конъюнктивных и дизъюнктивных формул. В этом смысле понятие квантора аналогично понятию суммы произвольного количества чисел и интеграла. Необходимость введения кванторов связана с тем, что область определения

предикатов может быть бесконечной. Например, если носитель модели состоит из элементов σ_i , то

$$\begin{aligned} ((\forall x P(x)) \Leftrightarrow (((P(\sigma_1) \wedge P(\sigma_1)) \wedge \dots) \wedge P(\sigma_i)) \wedge \dots)) \\ ((\forall x P(x)) \Leftrightarrow (((P(\sigma_1) \vee P(\sigma_2)) \vee \dots) \vee P(\sigma_i)) \vee \dots)) \end{aligned}$$

аналогично тому, как:

$$\begin{aligned} (\sum_{i=1}^n i = (1+2+\dots+n)) \\ (\int_a^b f(x) dx = \int_a^c f(x) dx + \int_c^b f(x) dx) \end{aligned}$$

Переменная (вхождение переменной) является в формуле свободной, если не существует кванторной формулы для этой переменной, которая является

подформулой рассматриваемой формулы (включающей этой вхождение).

Иначе переменная (вхождение переменной) называется связанной

(соответствующим квантором).

Рассмотрим модель, носитель которой имеет два элемента σ и τ . Тогда

следующая формула может в рамках $((\exists \vee \wedge R S (,,) () \chi \delta \lambda \chi))$ этой модели

следующие интерпретации (в этом случае, предикат S может характеризовать рефлексивное бинарное отношение).

Квантор – общее название для логических операций, ограничивающих область истинности какого-либо предиката

Кванторы:

⌚ $\forall x P(x) = P(x_1) \wedge P(x_2) \wedge \dots \wedge P(x_n)$

⌚ $\exists x P(x) = P(x_1) \vee P(x_2) \vee \dots \vee P(x_n)$

Квантор связывает переменные

Квантификация – приписывание квантора к формуле

Переменная свободная, если не существует кванторной формы для этой переменной, которая является подформулой рассматриваемой формулы.

Иначе – связанная

(Все ниже хуйня, но это все что есть в методе, по этому вопросу, чтобы вы туда не лезли, то есть если думаете, то есть пробежитесь глазами, но дальше ничего важного нет)

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

Для элементов σ и τ существуют различные модели, носитель которых

содержит только эти два элемента, но различающиеся сигнатурой. Например,

возможна модель, в которой могут быть построены следующие интерпретации.

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\perp$	$\perp$	$\perp$	$\bullet$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

Другими примерами моделей для элементов σ и τ являются следующие.

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\perp$	$\bullet$	$\bullet$	$\perp$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\perp$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\bullet$	$\perp$	$\bullet$	$\perp$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
--------	--------	---------	---------	---------	---------	---------

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\bullet$	$\perp$	$\bullet$	$\bullet$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\bullet$	$\bullet$	$\bullet$	$\perp$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\bullet$	$\bullet$	$\bullet$	$\bullet$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	σ	$\perp$	$\perp$	$\perp$	$\bullet$	$\bullet$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

χ	δ	$R(\chi, \delta, \sigma)$	$R(\chi, \delta, \tau)$	$(\exists \lambda R(\chi, \delta, \lambda))$	$S(\chi)$	$((\exists \lambda R(\chi, \delta, \lambda)) \vee S(\chi))$
σ	σ	$\perp$	$\bullet$	$\bullet$	$\perp$	$\bullet$
τ	σ	$\perp$	$\perp$	$\perp$	$\bullet$	$\bullet$
σ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
τ	τ	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

и т.д.

В дополнение к равносильным преобразованиям логики высказываний в логике предикатов рассматриваются следующие равносильные преобразования над кванторами.

Коммутативность.

$$((\forall x(\forall \lambda \alpha(x, \lambda))) \Leftrightarrow (\forall \lambda(\forall x \alpha(x, \lambda))))$$

$$((\exists x(\exists \lambda \alpha(x, \lambda))) \Leftrightarrow (\exists \lambda(\exists x \alpha(x, \lambda))))$$

Дистрибутивность.

$$((\forall x(\alpha(x) \wedge \beta(x))) \Leftrightarrow ((\forall x \alpha(x)) \wedge (\forall x \beta(x))))$$

$$((\exists x(\alpha(x) \vee \beta(x))) \Leftrightarrow ((\exists x \alpha(x)) \vee (\exists x \beta(x))))$$

Вынос констант (γ).

$$((\forall x(\alpha(x) \wedge \gamma)) \Leftrightarrow ((\forall x \alpha(x)) \wedge \gamma))$$

$$((\exists x(\alpha(x) \wedge \gamma)) \Leftrightarrow ((\exists x \alpha(x)) \wedge \gamma))$$

$$((\forall x(\alpha(x) \vee \gamma)) \Leftrightarrow ((\forall x \alpha(x)) \vee \gamma))$$

$$((\exists x(\alpha(x) \vee \gamma)) \Leftrightarrow ((\exists x \alpha(x)) \vee \gamma))$$

Двойственность кванторов.

$$((\neg(\forall x \alpha(x))) \Leftrightarrow (\exists x(\neg \alpha(x))))$$

$$((\neg(\exists x \alpha(x))) \Leftrightarrow (\forall x(\neg \alpha(x))))$$

20. Открытые и замкнутые формулы в языке логики предикатов

Формула открытая, если и только если в формуле существует хотя одна несвязанная переменная.

Замкнутая формула – формула, в которой все переменные связаны.

Терм – выражение формального языка, являющееся формальным именем объекта.

– символьное выражение $t(x_1, x_2, \dots, x_n)$, где t – имя («функциональная буква»), а $x_1, x_2, \dots, x_n$ – термы.

Терм определяется рекурсивно:

- ⌚ Всякая константа – терм
- ⌚ Всякая свободная переменная – терм
- ⌚ Если f_i – i -местная функциональная константа и $t_1, t_2, \dots, t_n$ – термы, то $f_i(t_1, t_2, \dots, t_n)$ – тоже терм
(Колмогоров с. 16)

Имена предметов и именные формы называют *термами*, высказывания и высказывательные формы — *формулами*.

Термами и формулами исчерпываются комбинации знаков, которым приписывается самостоятельный смысл. Иногда в математической логике термины «терм» и «формула» понимаются более узким образом, как комбинации знаков в некоторых точных логико-математических языках, например в так называемых языках первого порядка (см. гл. II. § 1).

21. Формальная теория (исчисление) предикатов первого порядка. Аксиомы и аксиомные схемы.

Формальные системы (ФС) – это совокупность чисто абстрактных объектов, не связанных с внешним миром, в котором представлены правила оперирования множеством символов в строго синтаксической трактовке без учета смыслового содержания, т.е. семантики. Формальная система задается:

- ⌚ Наличие конечного алфавита (словарь).
- ⌚ Правило построений формул.
- ⌚ Должно быть задано конечное число аксиом.
- ⌚ Правила вывода. Позволяют выводить теоремы из аксиом или других теорем.

Аксиома – это формула, считающейся истинной без доказательства. С помощью аксиомных схем задаются аксиомы. Отличие аксиомной схемы от аксиомы в том, что аксиомная схема записывается на некотором метаязыке, формулы которого содержат метапеременные. Любая аксиома может быть получена путем одновременной замены каждой метапеременной во всех ее вхождениях в аксиомную схему на конкретную формулу языка.

Например, аксиомные схемы, задающие правила для введения и удаления логических связок по отношению к формулам, являющимся подформулами этих аксиом:

	введение	удаление
$\rightarrow$	$(\alpha \rightarrow (\beta \rightarrow \alpha))$	$((\gamma \rightarrow \alpha) \rightarrow ((\gamma \rightarrow (\alpha \rightarrow \beta)) \rightarrow (\gamma \rightarrow \beta)))$
$\wedge$	$(\alpha \rightarrow (\beta \rightarrow (\alpha \wedge \beta)))$	$((\alpha \wedge \beta) \rightarrow \alpha)$ $((\alpha \wedge \beta) \rightarrow \beta)$
$\vee$	$(\alpha \rightarrow (\alpha \vee \beta))$ $(\alpha \rightarrow (\beta \vee \alpha))$	$((\alpha \rightarrow \gamma) \rightarrow ((\beta \rightarrow \gamma) \rightarrow ((\alpha \vee \beta) \rightarrow \gamma)))$
$\neg$	$((\alpha \rightarrow \beta) \rightarrow ((\alpha \rightarrow (\neg \beta)) \rightarrow (\neg \alpha)))$	$((\neg (\neg \alpha)) \rightarrow \alpha)$
$\sim$	$((\alpha \rightarrow \beta) \rightarrow ((\beta \rightarrow \alpha) \rightarrow (\alpha \sim \beta)))$	$((\alpha \sim \beta) \rightarrow (\alpha \rightarrow \beta))$ $((\alpha \sim \beta) \rightarrow (\beta \rightarrow \alpha))$

Множество аксиом является подмножеством формальной модели. Так как формальная модель может содержать бесконечное число формул, то правила вывода позволяют из аксиом вывести остальные формулы формальной модели. Одна и та же формальная модель может быть задана разными наборами аксиом и правил вывода.

Множество аксиом может быть бесконечным (в этом случае задается при помощи аксиомных схем), и множество правил вывода содержать только одно правило вывода, или – множество аксиом может быть пустым, а множество правил вывода может содержать несколько правил вывода.

Результат подстановки t вместо v в атомарную формулу F получается из F одновременной заменой всех вхождений v на t , где v – переменная, t – формула языка.

22. Формальная теория предикатов первого порядка. Правила вывода, теоремы и формальный вывод в логике предикатов.

Формальные системы (ФС) – это совокупность чисто абстрактных объектов, не связанных с внешним миром, в котором представлены правила оперирования множеством символов в строго синтаксической трактовке без учета смыслового содержания, т.е. семантики. Формальная система задается:

- ⌚ Наличие конечного алфавита (словарь).
- ⌚ Правило построений формул.
- ⌚ Должно быть задано конечное число аксиом.
- ⌚ Правила вывода. Позволяют выводить теоремы из аксиом или других теорем.

Над формулами формальной системы с помощью правил вывода задается отношение выводимости $\vdash$. Рассмотрим правило прямого заключения, описанное на метаязыке следующим образом:

Для любого множества G

$$G \subseteq L$$

и для любых формул α и β языка L , выполняется следующее:

$$((G \cup \{\alpha, (\alpha \rightarrow \beta)\}) \vdash (G \cup \{\beta\})),$$

На лекциях $\vdash$ обозначалось $\models$, кроме вышеприведённого свойства отношение вывода является транзитивным отношением и обладает следующим свойством:

$$((G \cup F) \vdash F),$$

Пример формального вывода (доказательства) теоремы $(P \rightarrow P)$ в заданном исчислении высказываний.

$$\langle (P \rightarrow (P \rightarrow P)), ((P \rightarrow (P \rightarrow P)) \rightarrow ((P \rightarrow ((P \rightarrow P) \rightarrow P)) \rightarrow (P \rightarrow P))), (P \rightarrow ((P \rightarrow P) \rightarrow P)), \\ ((P \rightarrow ((P \rightarrow P) \rightarrow P)) \rightarrow (P \rightarrow P)), (P \rightarrow P) \rangle$$

Рефлексивность отношения выводимости.

$$(G \vdash G)$$

Монотонность отношения выводимости.

$$((G \vdash E) \rightarrow ((G \cup F) \vdash E))$$

Транзитивность отношения выводимости.

$$(((G \vdash F) \wedge (F \vdash E)) \rightarrow (G \vdash E))$$

Содержательно каждое правило грамматики $\alpha \rightarrow \alpha\gamma\alpha$ имеет смысл подстановки. Например, строка α означает возможность замены метасимвола α на цепочку $\alpha\gamma\alpha$. Начав со стартового символа и пользуясь различными правилами грамматики, мы можем получать различные цепочки из символов, которые называются выводимыми цепочками. Если же метасимволов в цепочке не осталось, то процесс ее преобразования закончен и больше с цепочкой ничего сделать нельзя.

23 Метатеорема о полноте исчисления(логики) предикатов первого порядка. Обратная теорема.

полнота (это означает, что для любой замкнутой формулы выводима либо она сама, либо её отрицание);

Теорема Гёделя о полноте

Теорема Гёделя о полноте исчисления

предикатов устанавливает однозначную связь между логической истинностью высказывания и его выводимостью в логике первого порядка.

Формула является выводимой в исчислении предикатов первого порядка тогда и только тогда, когда она общезначима (истинна в любой интерпретации при любой подстановке).

Иными словами, если Φ - тождественно истинная формула

исчисления предикатов, то Φ доказуема в исчислении предикатов.

Доказательство

Из тождественной истинности Φ получаем, что множество $\{\neg\Phi\}$ не имеет модели. Из теоремы о существовании модели следует, что $\{\neg\Phi\}$ противоречиво, то есть $\neg\Phi \vdash$ - теорема исчисления предикатов. По правилу

$$\frac{\Gamma, \neg\Phi \vdash}{\Gamma \vdash \Phi}$$

вывода $\Gamma \vdash \Phi$ получаем, что Φ доказуема.

24 Метатеорема дедукции исчисления предикатов первого порядка. Обратная теорема.

Метатеорема - теорема относительно объектов (понятий, определений, аксиом, доказательств, правил вывода, теорем и др.) какой-либо научной теории (т. н. предметной, или объектной, теории), доказываемая средствами метатеории этой теории. Термин «метатеорема» употребляется преимущественно в применении к теоремам об объектах формализованных теорий (т. е. в случае, когда предметная теория является исчислением, или формальной системой).

Метатеорема дедукции: если из множества Γ и формулы A выводима формула B , то из множества Γ выводима формула $A \rightarrow B$, т.е.

$$\Gamma, A \vdash B \Rightarrow \Gamma \vdash A \rightarrow B$$

Следствие:

1. правило силлогизма: $A \rightarrow B, B \rightarrow C \vdash A \rightarrow C$

2. правило удаления средней посылки: $A \rightarrow (B \rightarrow C), B \vdash A \rightarrow C$

3. правило удаления крайней посылки: $(A \rightarrow B) \rightarrow C \vdash B \rightarrow C$

Хотя она и называется "теоремой" дедукции – это метод, который применяется для доказательства теорем. То есть, как говорят, метатеорема. Метатеорема дедукции (МТД): $G_1, G_2, \dots, G_{\beta-1}, G_{\beta} \vdash R$, то можно составить доказательство $G_1, G_2, \dots, G_{\beta-1} \vdash G_{\beta} \Rightarrow R$. Проще говоря, значок $\vdash$ можно переносить справа налево, на место запятой, оставляя вместо него на каждом шаге символ $\Rightarrow$:

$$G_1, G_2, G_3, G_4 \vdash R$$

$$G_1, G_2, G_3 \vdash G_4 \Rightarrow R$$

$$G_1, G_2 \vdash G_3 \Rightarrow (G_4 \Rightarrow R)$$

$$G_1 \vdash G_2 \Rightarrow (G_3 \Rightarrow (G_4 \Rightarrow R))$$

$$\vdash G_1 \Rightarrow (G_2 \Rightarrow (G_3 \Rightarrow (G_4 \Rightarrow R)))$$

Обратное движение также допустимо, причем, обоснование этого (в отличие от метатеоремы дедукции) элементарно. Пусть у нас имеется старое доказательство: $G_1, G_2, \dots, G_{\beta-1} \vdash G_{\beta} \Rightarrow R$. Построим на его основе новое доказательство: $G_1, G_2, \dots, G_{\beta-1}, G_{\beta} \vdash R$. Если R - аксиома, или одна из гипотез $G_1, G_2, \dots, G_{\beta}$, то новое доказательство получится сразу (длиной в 0

строк). Иначе возьмем старое доказательство целиком и добавим в его конец одну дополнительную строку: $G_{\beta}, G_{\beta} \Rightarrow R \vdash R$ // MP - старое доказательство плюс одна эта строка как раз и составят новое доказательство. Самое крайнее положение значка $\vdash$ при движении вправо - перед последней формулой R (далее modus ponens двигать значок не позволяет). Самое крайнее положение значка $\vdash$ при движении влево - с краю формулы. При движении значка вправо символ $\vdash$ должен ставится на место операции $\Rightarrow$, которая обрабатывается (согласно скобкам и приоритетам) самой последней. Если в формуле последней обрабатывается другая операция, то дальнейшее движение значка невозможно. При движении влево надо не забывать расставлять скобки, чтобы в образовавшейся формуле новый символ $\Rightarrow$ обрабатывался (согласно скобкам и приоритетам) самым последним.

Теорема дедукции.

$$(((G \cup \{\alpha\}) \vdash \{\beta\}) \rightarrow (G \vdash \{(\alpha \rightarrow \beta)\}))$$

Теорема обратная теореме дедукции.

$$((G \vdash \{(\alpha \rightarrow \beta)\}) \rightarrow ((G \cup \{\alpha\}) \vdash \{\beta\}))$$

Метатеоремы, формулирующие дополнительные правила вывода.

	введение	удаление
$\rightarrow$	$(((G \cup \{\alpha\}) \vdash \{\beta\}) \rightarrow (G \vdash \{(\alpha \rightarrow \beta)\}))$	$((G \cup \{\alpha, (\alpha \rightarrow \beta)\}) \vdash (G \cup \{\beta\}))$
$\wedge$	$((G \cup \{\alpha, \beta\}) \vdash (G \cup \{(\alpha \wedge \beta)\}))$	$((G \cup \{(\alpha \wedge \beta)\}) \vdash (G \cup \{\alpha\}))$ $((G \cup \{(\alpha \wedge \beta)\}) \vdash (G \cup \{\beta\}))$
$\vee$	$((G \cup \{\alpha\}) \vdash (G \cup \{(\alpha \vee \beta)\}))$ $((G \cup \{\alpha\}) \vdash (G \cup \{(\beta \vee \alpha)\}))$	$((G \cup \{(\alpha \vee \beta), (\alpha \rightarrow \gamma), (\beta \rightarrow \gamma)\}) \vdash (G \cup \{\gamma\}))$
$\neg$	$((G \cup \{(\alpha \rightarrow \beta), (\alpha \rightarrow (\neg \beta))\}) \vdash (G \cup \{(\neg \alpha)\}))$	$((G \cup \{(\neg(\neg \alpha))\}) \vdash (G \cup \{\alpha\}))$
$\sim$	$((G \cup \{(\alpha \rightarrow \beta), (\beta \rightarrow \alpha)\}) \vdash (G \cup \{(\alpha \sim \beta)\}))$	$((G \cup \{(\alpha \sim \beta)\}) \vdash (G \cup \{(\alpha \rightarrow \beta)\}))$ $((G \cup \{(\alpha \sim \beta)\}) \vdash (G \cup \{(\beta \rightarrow \alpha)\}))$

Отметим, что можно построить исчисление высказываний, используя в качестве аксиом пустое множество и только вышеперечисленные правила вывода.

25. Эрбрановой моделью сигнатуры W будем называть пару $M = \langle U, m \rangle$, где m – вероятность на $\hat{A}$. Функциональные символы интерпретируются на U обычным образом.

Эрбрановой моделью сигнатуры W будем называть вероятностную Эрбранову модель $M = \langle U, l \rangle$, где $l : \hat{A} \rightarrow \{0, 1\}$.

3.10.2. Теорема Эрбрана

Доказательство противоречивости формулы F основывается на использовании отрицания. Теорема Эрбрана гарантирует, что существующее противоречие может быть всегда достигнуто за конечное число шагов, каковы бы ни были значения истинности, даваемые функциям, присутствующим в гипотезах и заключениях.

Теперь приведем некоторые определения, которые потребуются нам в дальнейшем.

Для начала будем считать, что операции выполняются только над стандартными выражениями, представляющими собой дизъюнктивные формы C_i , которые называются предложениями:

$$C_i = (L_{i1} \vee L_{i2} \vee \dots \vee L_{in_i}).$$

Формула F представляет собой конъюнкцию предложений: $F = \bigwedge C_i$. Ниже мы увидим, что подобная стандартизация всегда возможна для формул первого порядка. Входящие в предложения C_i литералы L_{jk} не содержат символов $\wedge$, $\vee$, $\exists$, или $\supset$.

Пусть E представляет собой множество предложений, G_0 — множество констант, появляющихся в E , и для всякого натурального числа i множество G_i представляет собой объединение множества G_{i-1} и всех термов $f_k(t_1, t_2, \dots, t_{m_k})$, где f_k — функция m_k аргументов-термов t_j ($j = 1, 2, \dots, m_k$) из G_{i-1} . Пусть

$$G(E) = \bigcup_{i=0}^{\infty} G_i.$$

Множество $G(E)$ по определению является универсумом Эрбрана множества E . База Эрбрана $B(E)$ некоторого множества предложений E представляет собой множество всех *атомарных формул* множества E , т. е. таких формул, каждая из которых содержит либо не более одного предиката, либо отрицания предиката. Доказательство теоремы Эрбрана состоит в построении такого дерева A над элементами $B(E)$, что

- 1) E — корень дерева A ;
- 2) всякая вершина дерева A является подмножеством E , случайным образом означенным;

3) всякий путь, исходящий из корня дерева, не может содержать одновременно p и $\neg p$, где p — какая-либо формула.

Ассоциировать (сопоставить) какое-то конкретное значение истинности с формулами, построенными над $B(E)$, означает просто выбрать какой-то конкретный путь на A . Противоречие выявляется, когда приведенное выше условие 3 уже не удовлетворяется. Теорема Эрбрана гарантирует, что это достигается всегда, когда E невыполнимо (т. е. противоречиво).

Теорема Эрбрана (1930 г.). Необходимым и достаточным условием того, чтобы множество предложений E было невыполнимым, является существование конечного противоречивого множества полностью означенных подформул из E .

Это множество может быть построено в виде дерева описанного выше типа с помощью простого метода, связанного со следующим фундаментальным результатом.

Эрбранова модель. Эрбранова модель — модель логики для проверки выполнимости теорий (по конспекту).

26.

Одним из важнейших инструментов теории моделей является теорема компактности, доказанная Мальцевым, которая утверждает, что множество формул первого порядка имеет модель тогда и только тогда, когда модель имеет каждое конечное подмножество этого множества формул.

Название теоремы связано с тем, что она может быть сформулирована как утверждение о компактности стоуновского пространства.

Из теоремы компактности следует, что некоторые понятия не являются выразимыми в логике первого порядка. Например, понятия конечности или счётности не могут быть выражены никакими формулами первого порядка и даже их множествами: если множество формул имеет сколь угодно большие конечные модели, то оно имеет и бесконечную модель. Аналогично, теория, имеющая бесконечную модель, мощность которой не меньше мощности сигнатуры, имеет модели и любой большей мощности.

Теорема компактности находит применение для конструирования нестандартных моделей классических теорий, например, элементарной арифметики или математического анализа.

27. Метатеоремы Лёвенгейма-Сколема для исчисления предикатов первого порядка.

Теорема Лёвенгейма-Сколема : Любое конечное или счётное выполнимое множество формул исчисления предикатов допускает конечную или счётную модель.

Другое определение, следствие из теоремы Геделя о полноте:

Следствие. (Теорема Сколема-Левенгейма, 1919, 1915 гг.) Если теория K первого порядка имеет какую-нибудь модель, то она имеет и счетную модель.

Доказательство. Если K имеет модель, то K непротиворечива. Следовательно, K имеет счетную модель.

28. Предварённая нормальная форма формул языка логики предикатов.

Так как операции конъюнкции, дизъюнкции и отрицания образуют логический базис, то любая истинностная функция является их суперпозицией, то есть может быть выражена через эти операции.

Следовательно, для любой истинностной функции можно построить обозначающую ее формулу языка логики высказываний. В связи с этим разделяют классы логических формул, называемых нормальными формами. Различают дизъюнктивную нормальную форму и конъюнктивную нормальную форму.

Элементарная конъюнкция – формула, подформулами которой являются только конъюнктивные формулы, либо – атомарные формулы, либо отрицания атомарных формул.

Элементарная дизъюнкция – формула, подформулами которой являются только дизъюнктивные формулы, либо – атомарные формулы, либо отрицания атомарных формул.

ДНФ – формула, подформулами которой являются дизъюнктивные формулы, либо элементарные конъюнкции.

КНФ – формула, подформулами которой являются конъюнктивные формулы, либо – элементарные дизъюнкции.

Предварённой нормальной конъюнктивной (дизъюнктивной) формой называется формула, в которой нет некванторной подформулы, которая имеет кванторную подформулу, и в которой присутствует в качестве подформулы КНФ (ДНФ), которая не является подформулой формулы, не являющейся кванторной формулой или КНФ (ДНФ). Другими словами, предварённая нормальная форма – это формула, у которой все кванторы записаны слева, а оставшаяся подформула является КНФ(ДНФ), не включающей кванторов.

Говорят, что предваренная формула является Σ_n - формулой, если ее кванторная приставка содержит n групп кванторов,

причем первыми стоят кванторы существования. Если первыми стоят кванторы всеобщности, говорят о классе Π_n . (Аналогичные обозначения используются в теории алгоритмов для классификации арифметических множеств). Пример: формула $\forall x \exists y \exists z \forall u (A(x, u, z) \rightarrow B(z, u))$ принадлежит классу Π_3 , формула $\exists u \forall v C(u, v)$ принадлежит классу Σ_2 , а формула $\forall x (A(x) \rightarrow \exists y B(x, y))$ вообще не находится в предваренной нормальной форме.

29. Сколемовская стандартная форма формул языка логики предикатов.

Сколемовская(стандартная) форма получается из предваренной нормальной, путём преобразования последней и записью в виде текста расширения рассмотренного языка предикатов, допускающего константы и функциональные термы в качестве аргументов предикатов.

Строится сколемовская стандартная форма следующим образом.

Если предваренная нормальная форма является кванторной формулой существования, то каждая во всех вхождениях переменная, связанная

квантором заменяется константой, которая отсутствует в формуле, и процесс повторяется для оставшейся подформулы.

Далее, если квантор существования расположен после кванторов общности,

то соответствующая переменная, связанная квантором существования

заменяется отсутствующим в исходной формуле функциональным термом,

список аргументов в котором содержит все переменные, связанные расположенными до рассматриваемого квантора существования кванторами

общности. Все кванторы общности исключаются из исходной формулы.

Рассмотрим следующий пример. Пусть есть формула

$$\left(\exists \chi \left(\forall \lambda \left(\forall \delta \left(\exists \gamma \left(\forall \sigma P(\chi, \lambda, \delta, \gamma, \sigma) \right) \right) \right) \right) \right)$$

, тогда её
сколемовская стандартная форма выглядит следующим
образом.

$$P(a, \lambda, \delta, \varphi(\langle \lambda, \delta \rangle), \sigma)$$

Использование сколемовских стандартных форм(ССФ)
позволяет упростить
применение метода резолюций к логике предикатов.

30. Унификаторы (НОУ) и унификация формул языка логики предикатов.

Задача унификации: По двум описаниям X и Y определить, можно ли найти объект Z, который удовлетворяет обоим описаниям. Обычно уточняется как задача нахождения по данным двум термам, содержащим переменные, такой подстановки термов вместо переменных, которая превратила бы исходные термы в идентичные. В том случае, когда такая подстановка для термов существует, она называется *унификатором*, а термы называются *унифицируемыми*. Если термы унифицируемы, то у них может быть много

унификаторов, но всегда существует и единствен **наибольший общий унификатор**, из которого с помощью композиций подстановок можно получить все другие унификаторы. Этот унификатор, обладает таким свойством, что для всякого другого унификатора q для того же множества выражений существует такая подстановка t , что $q = qt$, где qt - композиция подстановок q и t .

Унификация (сопоставление) является основной операцией в методе резолюций. Пусть даны два дизъюнкта $C_1 = \{l_1, \dots\}$ и $C_2 = \{l_2, \dots\}$, принадлежащие множеству S . Предположим, что литеры l_1 и l_2 являются унифицируемыми, т. е. обладают общей фундаментальной конкретизацией. Из каждой пары фундаментальных конкретизаций $C'_1 = \{l'_1, \dots\}$ и $C'_2 = \{\neg l'_2, \dots\}$, таких, что $l_1 = l'_1 = l'_2$ получается резольвента $R = (C'_1 \setminus \{l'_1\}) \cup (C'_2 \setminus \{\neg l'_2\})$. Каждый дизъюнкт такого типа является логическим следствием из C_1 и C_2 . Задача состоит в том, чтобы найти такой дизъюнкт R , фундаментальные конкретизации которого были бы в точности конкретизациями того же самого типа, как и для R' . Обозначим через lU наибольшую нижнюю границу пары $\{l_1, l_2\}$ относительно порядка $<$. Наиболее общим унификатором литер l_1 и l_2 , будем называть подстановку sU , такую, что $sU[l_1] = sU[l_2] = lU$. Резольвентным дизъюнктом (резольвентой) R с требуемым свойством будет следующее выражение: $R = (\sigma_U[C_1] \setminus \{l_U\}) \cup (\sigma_U[C_2] \setminus \{\neg l_U\})$.

То есть, **унификация** - процедура подстановки термов в два логических выражения вместо переменных. Термы подбираются таким образом, что при замене ими одноименных в двух выражениях переменных оба выражения становятся идентичными. Сама подстановка называется унификатором. У. используется при логическом выводе в методе резолюций.

31. Правило резолюции в логике предикатов.

Правило резолюций – это правило вывода, восходящее к методу доказательства теорем через поиск противоречий; используется в логике высказываний и логике предикатов первого порядка. Правило резолюций, применяемое последовательно для списка резольвент, позволяет ответить на вопрос, существует ли в исходном множестве логических выражений противоречие. Правило разработано Джоном Аланом Робинсоном в 1965.

Пусть C_1 и C_2 - два предложения в исчислении высказываний, и пусть $C_1 = P \vee C'_1$, а $C_2 = \neg P \vee C'_2$, где P - пропозициональная переменная, а C'_1 и C'_2 - любые предложения (в частности, может быть, пустые или состоящие только из одного литерала).

Правило вывода $\frac{C_1, C_2}{C'_1 \vee C'_2} R$ называется **правилом резолюции**.

Предложения C_1 и C_2 называются резольвируемыми (или родительскими), предложение $C'_1 \vee C'_2$ - резольвентой, а формулы P и $\neg P$ - контрарными литералами. Метод резолюций является обобщением метода "доказательства от противного". Вместо того чтобы пытаться вывести некоторую формулу-гипотезу из имеющегося непротиворечивого множества аксиом, мы добавляем отрицание нашей формулы к множеству аксиом и пытаемся вывести из него противоречие. Пользуясь законом исключенного третьего мы приходим к выводу, что исходная формула была выводима из множества аксиом.

32.Хорновские дизъюнкты.

Хорновский дизъюнкт - это дизъюнкт, содержащий не более одной литеры без отрицания.

Дизъюнкт - это совокупность литер, одни из которых не содержат, а другие содержат знак отрицания. Если первые записывать через точку с запятой слева от знака ":-", а вторые - без знака отрицания через запятую справа от знака ":-" и в конце каждого дизъюнкта ставить точку, то получим выражение вида: $A_1;A_2;...;A_n:-B_1,B_2,...,B_m$

Существует **два** вида хорновских дизъюнктов:

- ⌚ **Хорновский дизъюнкт с заголовком** - это дизъюнкт, содержащий одну литеру без отрицания. Он может содержать одну или несколько литер с отрицанием или не содержать их вообще, например: $A:-B,C,...,D$; $A:-B$; $A:-$. Последний из приведенных дизъюнктов можно записывать без знака ":-", то есть в виде A .
- ⌚ **Хорновский дизъюнкт без заголовка** - это дизъюнкт, не содержащий литер без отрицания, например: $:-B,C,...,D$; $:-A$.

В результате применения метода резолюций к двум **хорновским дизъюнктам** с заголовками вновь получается **хорновский дизъюнкт** с заголовком.

Пустой дизъюнкт не имеет заголовка. Следовательно, если все дизъюнкты имеют заголовки, то из них могут быть получены только дизъюнкты с заголовками. Таким образом, для того чтобы из исходных дизъюнктов, отрицая доказываемое высказывание, можно было бы вывести пустой дизъюнкт, необходимо наличие, по крайней мере, одного дизъюнкта без заголовка. Если среди исходных имеется несколько дизъюнктов без заголовка, то доказательство каждого нового дизъюнкта методом резолюций может быть преобразовано в доказательство, в котором используется не более чем один дизъюнкт без заголовка. Если пустой дизъюнкт следует из заданного множества исходных, то он следует и из его подмножества, содержащего дизъюнкты с заголовками и не более одного дизъюнкта без заголовка. Любая задача,

которая может быть выражена с помощью **хорновских дизъюнктов**, должна иметь только один дизъюнкт без заголовка (целевой); все остальные дизъюнкты должны быть с заголовками (гипотезы). **Хорновские дизъюнкты** были выбраны в качестве основы для создания программных систем автоматического доказательства теорем.

33) Прикладные исчисления предикатов и их отличие от чистого исчисления предикатов. Логические и собственные аксиомы. Примеры.

Предикат – высказывательная функция, определенная на некотором множестве M , т.е. такая n -местная функция P , которая каждому упорядоченному набору $\langle a_1, a_2, \dots, a_n \rangle$ элементов множества M сопоставляет некоторое высказывание, обозначаемое $P(a_1, a_2, \dots, a_n)$.

Исчисление предикатов — формальное исчисление, допускающее высказывания относительно переменных, фиксированных функций, и предикатов. Расширяет логику высказываний.

Прикладные исчисления, в отличие от «чистых» исчислений, содержат дополнительные предикатные константы в языке и дополнительные аксиомы, определяющие свойства этих констант. Прикладные исчисления можно разбить на абстрактные и предметные, т.е. те, которые описывают свойства «чистых» математических абстракций и те, которые описывают свойства физических объектов. Предметные прикладные исчисления обычно строятся с помощью языков логического программирования в рамках некоторой программной системы, решающей ту или иную прикладную задачу. Абстрактные прикладные исчисления встречаются чаще.

ЛОГИЧЕСКИЕ И СОБСТВЕННЫЕ АКСИОМЫ

1. Обычно аксиомы делятся на два вида: логические аксиомы (общие для целого класса формальных теорий) и

нелогические или собственные аксиомы (определяющие специфику и содержание конкретной теории).

2. Логические аксиомы выражают общие законы формальной логики, которые должны выполняться в любой научной теории. Логические аксиомы, в отличие от нелогических, истинны по смыслу логических символов, т.е. они истинны в каждой интерпретации любого множества имен. Логическая аксиома не содержит других метаконструкций, кроме синтаксических переменных.
3. Нелогические аксиомы должны обозначать какие-то специальные законы и принципы, характерные только для данной научной теории. Нелогические аксиомы считаются правильными, если их достоверность установлена на основе интуитивного критерия.
4. Аксиомы, пример:

- | | | |
|----|-------------------|-------------------|
| 1. | $\neg(\neg x)=x,$ | $x=\neg(\neg x);$ |
| 2. | | $x*(\neg x)=0;$ |
| 3. | | $x+1=1;$ |
| 4. | $x+x=x,$ | $x=x+x+x;$ |
| 5. | | $x+0=x;$ |
| 6. | $x*x=x,$ | $x=x*x*x;$ |
| 7. | | $x*0=0;$ |
| 8. | | $x*1=x;$ |

34) Язык SCL. Ключевые узлы, логические связки. Логический и фактографические уровни scl-текста. Примеры.

Формальный логический язык SCL построен на базе языка SC как его подязык путем фиксации определенного набора специальных ключевых узлов, т.е. узлов, семантика которых должна быть априори известна и согласована.

Основные ключевые узлы языка SCL: множество атомарных логических формул, множество конъюнктивных логических формул, множество дизъюнктивных формул, множество альтернативных формул, множество импликативных формул, (посылка импликативной логической формулы, следствие импликативной логической формулы, множество логических формул об эквивалентности, множество негативных логических формул, множество формул о существовании, множество формул о всеобщности. Пособие МОИИ стр. 125-126.

Связка – это множество, знак которого принадлежит какому-

либо нормализованному отношению. Простейшим видом связок являются атомарные логические связки, элементами которых являются логические константы и/или переменные.

В логике *логическими связками* называют действия, вследствие которых порождаются новые понятия, возможно с использованием уже существующих. В качестве основных обычно называют конъюнкцию, дизъюнкцию, импликацию, отрицание.

Трактовка атомарных логических формул как множеств, элементами которых являются элементарные составляющие логических формул (константы и переменные), явное введение знаков для всех логических формул, входящих в состав логического текста, и, наконец, трактовка неатомарных (сложных) логических формул как множеств, элементами которых являются знаки логических формул, которые входят в состав этих неатомарных логических формул, – все это позволяет использовать фактографический язык SCB для представления логических формул и для описания соотношений между ними. Это **фактографический уровень SCL**. Единственная особенность такого использования языка SCB заключается в том, что здесь приходится иметь дело не только с множествами, элементами которых являются знаки множеств (т.е. нормализованными множествами), но и с множествами, среди элементов которых встречаются переменные, которые, строго говоря, знаками множеств не являются. Это **логический уровень SCL**.

35Язык SCL. Представление открытых и закрытых формул.
Правила и способы бескванторной записи формул.
Примеры.

Формулы содержащие свободные переменные называются открытыми. А формулы, которые не содержат свободные переменные называются закрытыми.

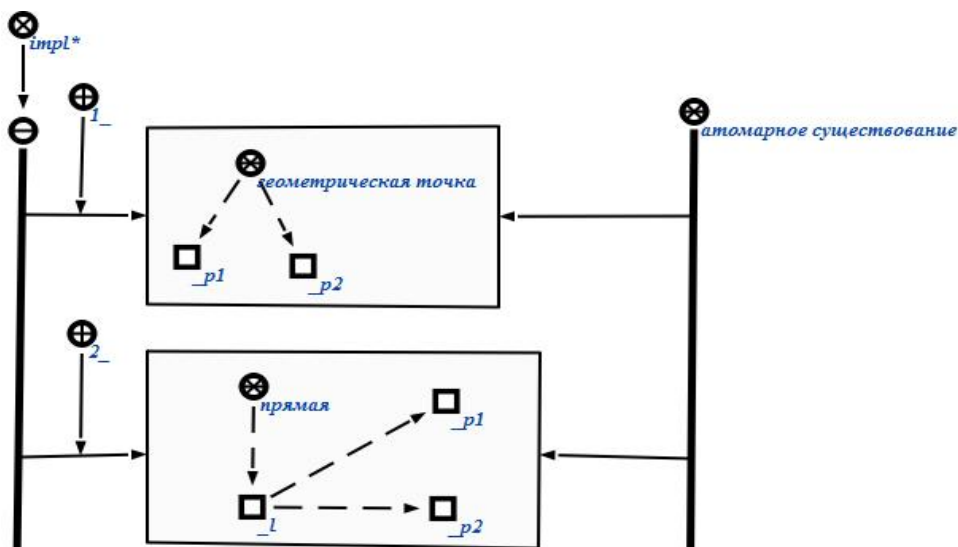
Примеры закрытых формул:

$P, (\forall z)(R(z)), (\exists x)(\forall y)(P(x,y)), (\forall x)(Q(x)) \rightarrow \neg(\forall x)(\exists x)(R(x,y)).$

Бескванторная запись логических утверждений

- ⌚ Квантор общности можно явно не обозначать, если какая-то переменная встречается в левой и правой части импликации (эквиваленции) и не попала выше в область действия других кванторов
- ⌚ Квантор существования можно не использовать, если переменная входит в атомарную формулу, которая является элементом множества «атомарное существование», и переменная не попала выше в область действия других кванторов

$\forall p1 \forall p2 ((Point(p1) \& Point(p2)) \rightarrow \exists (Line(l) \& Belonging(p1, l) \& Belonging(p2, l)))$, где
 Point (p) – p является геометрической точкой
 Line (l) – l является прямой



36. Язык SCL. Представление теорий. Связь теории с предметной областью. Соответствие и переходы между языком предикатов первого порядка и языком SCL. Примеры.

Формальный логический язык **SCL** построен на базе языка SC как его подязык путем фиксации определенного набора специальных ключевых узлов, т.е. узлов, семантика которых должна быть априори известна и согласована.

Представление теорий:



Связь теории с предметной областью организовывается через атрибутивное отношение «предметная область_», которое связывает теорию с реляционной структурой, описывающей предметную область, являющейся моделью предметной области.

Соответствие и переходы:

$\forall$ - всеобщность*

$\exists$ - существование*

$\wedge$ - конъюнкция*

$\vee$ - дизъюнкция*

$\rightarrow$ - эквиваленция*

$\sim$ - импликация*

Пример:

$\forall x P(x)$



37. Логическое программирование, язык ПРОЛОГ. Структура ПРОЛОГ программы, соответствие и отличие от языка логики предикатов. Термы и их виды в языке ПРОЛОГ.

Логическое программирование — парадигма программирования, основанная на автоматическом доказательстве теорем, а также раздел дискретной математики, изучающий принципы логического вывода информации на основе заданных фактов и правил вывода. Логическое программирование основано на теории и аппарате математической логики с использованием математических принципов резолюций.

Структура программы на Турбо Прологе

Программа на Турбо Прологе состоит из следующих семи разделов:

- ⌚ *директивы компилятора ;*
- ⌚ **CONSTANTS** - раздел описания *констант ;*
- ⌚ **DOMAINS** - раздел описания *доменов ;*
- ⌚ **DATABASE** - раздел описания предикатов внутренней базы данных;
- ⌚ **PREDICATES** - раздел описания предикатов;
- ⌚ **CLAUSES** - раздел описания *предложений ;*
- ⌚ **GOAL** - раздел описания *внутренней цели.*

В программе не обязательно должны быть все эти разделы. Так, например, она может состоять из одного описания цели:

GOAL
`write("hello"),readchar(_).`

Эта программа, вполне в императивном духе, выведет сообщение (с помощью стандартного предиката **write**) и будет ожидать нажатия пользователем любой клавиши (стандартный предикат **readchar** читает символ).

Однако, как правило, программа содержит, по меньшей мере, разделы **PREDICATES** и **CLAUSES**.

Если программа запускается в среде разработки Турбо Пролога, то раздел **GOAL** необязателен. При написании же программы,

не зависящей от среды разработки, в ней необходимо указать *внутреннюю цель*.

В программе может быть несколько разделов описаний **DOMAINS**, **PREDICATES**, **DATABASE** и **CLAUSES**. Однако разделов **GOAL** не может быть в программе более одного.

Порядок разделов может быть произвольным, но при этом *константы*, *домены* и предикаты должны быть определены до их использования. Однако в разделе **DOMAINS** можно ссылаться на *домены*, которые будут объявлены позже.

Предикаты выражают некоторое условие, например объект зеленый или число простое, естественно что условия имеют входные параметры.

Например *green_object(Object),prime_number(Number)* . Сколько в предикате параметров, такова и арность предиката.

Термы

Программа на Прологе описывает отношения, определяемые с помощью предложений. Как и в любом другом языке, ориентированном на **символьные вычисления**, предложения выстраиваются из термов, которые в свою очередь подразделяются на атомы, числа, переменные и структуры. Атом записывается со строчной буквы или заключается в кавычки, когда требуется запись с прописной буквы.

```
atom  
'Atom'
```

Переменные, записывающиеся с прописной буквы, отличаются от переменных в процедурных языках программирования, они не связаны с конкретной ячейкой памяти, а скорее ближе к математической переменной.

```
X is 2 + 2.
```

Структуры представляют собой совокупности термов, заключенные в круглые скобки, в том числе и другие структуры. Структура обозначается именем (функтором), которое располагается перед круглыми скобками.

```
book( 'Название', '2009', 'Спб', authors( 'Первый автор', 'Второй автор' ) ).
```

Ещё одной конструкцией являются списки, элементы которых заключаются в квадратные скобки. В основе списков в Пролог лежат **связные списки**.

```
List = [ a, b, [ c, d ], e ].
```

38. Логическое программирование, язык ПРОЛОГ. Встроенные предикаты языка ПРОЛОГ.

ИНФА ВЗЯТА: <http://wm-help.net/lib/b/book/27531012/133>.

Имеются два основных встроенных предиката для ввода новых утверждений: **consult** и **reconsult**.

В языке имеются два предиката, позволяющих удобно и явным образом определить случаи, когда целевое утверждение считается согласованным и когда несогласованным. Это предикаты **true** и **fail**.

Если вам надо определить предикаты, которые будут использоваться с аргументами различных типов, то полезно иметь возможность выделять в определении предиката ситуации, соответствующие каждому из возможных типов. В простейшем случае может понадобиться применять разные утверждения в зависимости от того, является ли аргумент целым числом или атомом. Разные утверждения могут потребоваться и для конкретизированных и неконкретизированных аргументов. Приводимые ниже предикаты позволяют программисту включить в утверждения эти дополнительные условия.

```
var(X)
nonvar(X)
atom(X)
integer(X)
atomic(X)
```

предикаты для работы со структурами произвольного вида:

```
functor(T,F,N)
```

```
⌚ arg(N,T,A)  
⌚ X=..L  
⌚ name(A,L)
```

Предикат «!» устраняет возможности для повторного согласования целевых утверждений, а предикат **repeat** создает новые альтернативы там, где их не было ранее.

В правилах и вопросах вида **X:-Y** или **?-Y** терм, появляющийся на месте **Y**, может состоять из единственного целевого утверждения либо представлять конъюнкцию целевых утверждений или их дизъюнкцию. Более того, можно употреблять в качестве целевых утверждений **переменные** и успешно доказывать согласованность целевого утверждения, когда целевое утверждение в действительности не согласуется, используя для этого предикат **not**. Предикаты, представленные в этом разделе, позволяют реализовать эти сложные способы выражения целевых утверждений.

В этом разделе коротко рассматриваются различные встроенные предикаты, используемые для проверки равенства элементов и позволяющие делать их равными.

- ⌚ X=Y
- ⌚ X=Y
- ⌚ X==Y
- ⌚ X == Y

Предикаты для ввода и вывода литер и термов обсуждались в гл. 5. Здесь мы резюмируем наши знания о каждом из этих предикатов.

- ⌚ get0(X)
- ⌚ get(X)
- ⌚ skip(X)
- ⌚ read(X)
- ⌚ put(X)
- ⌚ nl
- ⌚ tab(X)
- ⌚ write(X)
- ⌚ display(X)

⌚ [op\(X,Y,Z\)](#)

Предикаты для изменения текущего входного и текущего выходного потоков данных были введены в гл. 5. Здесь мы резюмируем наши знания о каждом из этих предикатов.

⌚ [see\(X\)](#)
⌚ [seeing\(X\)](#)
⌚ [seen](#)
⌚ [tell\(X\)](#)
⌚ [telling\(X\)](#)
⌚ [told](#)

Арифметические возможности языка Пролог первоначально обсуждались в разд. 2.5. Здесь мы подытожим наши знания об использовании предиката '**is**' и о том, какие имеются функторы для формирований арифметических выражений.

⌚ [X is Y](#)
⌚ [X + Y](#)
⌚ [X - Y](#)
⌚ [X * Y](#)
⌚ [X / Y](#)
⌚ [X mod Y](#)

В Прологе имеются шесть предикатов для сравнения целых чисел. Эти предикаты первоначально были представлены в разд. 2.5, когда мы обсуждали арифметические возможности языка. Каждый такой предикат записывается как инфиксный оператор, имеющий два аргумента.

⌚ [X = Y](#)
⌚ [X = Y](#)
⌚ [X < Y](#)
⌚ [X > Y](#)
⌚ [X >= Y](#)
⌚ [X =< Y](#)

В этом разделе описаны встроенные предикаты, которые позволяют наблюдать за выполнением вашей программы. Здесь мы лишь опишем эти встроенные предикаты, а более

подробное обсуждение отладки и трассировки программ содержится в гл. 8.

- 🕒 [trace](#)
- 🕒 [notrace](#)
- 🕒 [spy P](#)
- 🕒 [debugging](#)
- 🕒 [nodebug](#)
- 🕒 [nospy](#)